OpenFOAM Simulation Guide

Hydrogen–Methane T-Junction DDES

Reproducing Eames et al. (2022) in OpenFOAM

Yash

April 18, 2026

Contents

1 Introduction **3**

1.1 Physical Problem . . . 3

1.2 Objectives of the Simulation . . . 3

1.3 Fluent ↔ Paper ↔ OpenFOAM . . . 4

1.4 Case Directory Layout . . . 4

2 Governing Equations **6**

2.1 Continuity . . . 6

2.2 Momentum (with Buoyant Pressure Decomposition) . . . 6

2.3 Species Transport . . . 6

2.4 Sensible Enthalpy . . . 7

2.5 Closure: Turbulence Model . . . 7

3 Geometry and Meshing **8**

3.1 Geometry Definition . . . 8

3.2 Step 1: Generate the STL . . . 8

3.3 Step 2: Background Mesh (`blockMesh`) . . . 9

3.4 Steps 3–4: Features + `snappyHexMesh` . . . 9

3.4.1 Boundary Patches . . . 9

4 Physical Models and Properties **10**

4.1 Turbulence Model (`constant/momentumTransport`) . . . 10

4.1.1 Turbulent Schmidt Number . . . 10

4.2 Thermophysical Properties . . . 11

4.2.1 Species Properties (NIST / Paper Verification) . . . 11

4.3 Chemistry & Combustion (Disabled) . . . 11

4.4 Gravity . . . 12

5 Boundary and Initial Conditions **13**

5.1 Power-Law Inlet Velocity Profile . . . 13

5.1.1 Branch Velocity Derivation . . . 14

5.2 Pressure (`0/p` and `0/p_rgh`) . . . 14

5.3 Temperature, Species, Turbulence . . . 15

5.4 Initialisation (`setFieldsDict`) . . . 15

6 Solution Methods and Controls **16**
6.1 Pressure–Velocity Coupling 16
6.2 Discretisation Schemes (`system/fvSchemes`) 16
6.3 Under-Relaxation Factors 17

7 Run Setup and Monitors **18**
7.1 Time-Stepping 18
7.2 Monitors (Function Objects) 18
7.2.1 Residuals, Courant Number, y+ 18
7.2.2 Mass Balance 18
7.2.3 Mixing Statistics 19
7.2.4 Wall Probes (Top + Bottom + Sides) 19
7.2.5 Hydrogen Volume Fraction (Coded Field) 19
7.2.6 Time-Averaged Fields 20
7.3 Full Workflow 20

8 Validation and Expected Results **21**
8.1 Validation Strategy 21
8.2 Expected Behaviour (Production Run, $\mathcal{D} = 0.2$) 21
8.3 Paper Physical Findings to Reproduce 22

9 Post-Processing **24**
9.1 In-Run Outputs 24
9.2 Paraview Workflow 24
9.2.1 Reproducing Paper Figures 24
9.3 Coefficient of Variation (COV) 25
9.4 Mesh Independence (GCI) 25

10 Quick-Start Summary **26**

Chapter 1

Introduction

This guide documents a *reference* OpenFOAM setup that reproduces the transient DDES simulation of hydrogen injection into a methane-filled T-junction reported by **Eames, Austin & Wojcik** (*International Journal of Hydrogen Energy*, 47:25745–25754, 2022). Every dictionary file, boundary condition and solver setting has been chosen to match either the paper directly or the companion Fluent configuration (where the Fluent and OpenFOAM setups are intended to be numerically equivalent).

1.1 Physical Problem

Pure hydrogen is injected at high pressure into a methane natural-gas pipeline through a 90° T-junction. Two aspects make the flow unusual:

1. The density ratio $\rho_{\mathrm{CH_4}}/\rho_{\mathrm{H_2}} \approx 8$ at 6.9 MPa — far larger than typical single-phase flows — generates strong **buoyancy-driven stratification**.

2. The low inertia of the H_2 branch stream leads to weak jet penetration, concentrating hydrogen against the pipe wall rather than mixing into the bulk.

The engineering consequence is **hydrogen embrittlement**: localised, near-wall H_2 concentrations much higher than the nominal blend fraction can accelerate fatigue crack growth in high tensile carbon steels.

1.2 Objectives of the Simulation

Following the paper, the simulation must resolve:

- The **time-averaged wall H_2 volume fraction** $\overline{V_R}$ as a function of downstream distance z/d_1, for multiple dilution factors.

- The **maximum vertical-plane H_2 concentration** V_{Rm} and its decay with z.

- The **stratification asymmetry** between the upper and lower pipe walls.

- Ultimately, the ratio $\mathcal{R} = \overline{V_{Rm}}/V_{R,T}$ at $z/d_1 = 8$, which the paper shows varies from 4.4 to 1.6 across the blend-fraction targets of 5–20 %.

1.3 Fluent ↔ Paper ↔ OpenFOAM

Table 1.1: Cross-reference of the three equivalent setups.

Setting	**Fluent (companion guide)**	**Eames 2022**	**OpenFOAM (this guide)**
Solver	Pressure-based transient	`rhoReactingBuoyantFoam`	`rhoReactingBuoyantFoam`
Turbulence	SA-DDES + curv. corr.	SA-DDES	`SpalartAllmarasDDES` + curv. corr.
P–V coupling	PISO	(OpenFOAM PIMPLE)	PIMPLE, `nOuterCorrectors 1`
Momentum scheme	Bounded Central Differencing	Low-dissipation central	`Gauss LUST`
Transient	Bounded 2nd-order implicit	2nd-order	`backward`
Inlet profile	Uniform (implicit)	**Power-law** $u_r = u^*(1-(2r/d^*)^2)^{1/7}$	`codedFixedValue` **power-law,** n=7
Gravity / buoyancy	Enabled, $g_y = -9.81$	Enabled	`constant/g`, $g_y = -9.81$
Mesh	ANSYS Meshing (2.8–4.8 M cells)	`snappyHexMesh` (2.8–4.8 M)	`blockMesh` + `snappyHexMesh`

Why this solver? Eames et al. use `rhoReactingBuoyantFoam` *specifically* because it solves for $p_{\mathrm{rgh}} = p - \rho\,\mathbf{g}\cdot\mathbf{h}$ rather than p directly. Removing the hydrostatic component from the momentum equation is essential for a flow where buoyancy is the dominant driver of the key physical result (stratification). Using the simpler `reactingFoam` would introduce spurious numerical noise in the pressure field and degrade the stratification prediction.

1.4 Case Directory Layout

```
openfoam_case/
  0/                    Initial & boundary conditions
    U  p  p_rgh  T  H2  CH4  nuTilda  nut  alphat
  constant/
```

```
  g                                gravity vector
  momentumTransport                turbulence (SA-DDES)
  turbulenceProperties             Schmidt number
  thermophysicalProperties         mixture/species
  chemistryProperties              chemistry OFF
  combustionProperties             combustion OFF
  reactions                        empty
  triSurface/                      STL geometry
system/
  controlDict                      run control + monitors
  fvSchemes                        discretisation
  fvSolution                       linear solvers, PIMPLE
  blockMeshDict                    background mesh
  snappyHexMeshDict                conformal mesh
  surfaceFeatureExtractDict        feature extraction
  setFieldsDict                    branch H2 patching
  decomposeParDict                 parallel decomposition
Allrun                             end-to-end pipeline
Allclean                           cleanup
generateSTL.py                     STL generator
```

Chapter 2

Governing Equations

These are the equations actually solved by `rhoReactingBuoyantFoam`, identical to those of the paper.

2.1 Continuity

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0$$

2.2 Momentum (with Buoyant Pressure Decomposition)

The solver writes $p = p_{\mathrm{rgh}} + \rho\, \mathbf{g} \cdot \mathbf{h}$ and advances p_{rgh} instead of p:

$$\frac{\partial(\rho u_i)}{\partial t} + \frac{\partial(\rho u_i u_j)}{\partial x_j} = -\frac{\partial p_{\mathrm{rgh}}}{\partial x_i} - (\mathbf{g} \cdot \mathbf{h}) \frac{\partial \rho}{\partial x_i} + \frac{\partial}{\partial x_j} \left[\mu_e \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right) \right] + \rho g_i$$

The second term on the right absorbs the hydrostatic gradient, so p_{rgh} has only the dynamic contribution.

2.3 Species Transport

For each species $k \in \{\mathrm{H}_2, \mathrm{CH}_4\}$:

$$\frac{\partial(\rho Y_k)}{\partial t} + \nabla \cdot (\rho Y_k \mathbf{u}) = \nabla \cdot (\mu_e \nabla Y_k)$$

CH_4 is the **inert (balance) species**, so $Y_{\mathrm{CH}_4} = 1 - Y_{\mathrm{H}_2}$ everywhere.

2.4 Sensible Enthalpy

$$\frac{\partial(\rho h)}{\partial t} + \nabla\cdot(\rho \mathbf{u} h) + \frac{\partial(\rho K)}{\partial t} + \nabla\cdot(\rho \mathbf{u} K) - \frac{\partial p}{\partial t} = \nabla\cdot(\mu_e \nabla h)$$

with $K = \frac{1}{2}|\mathbf{u}|^2$ the specific kinetic energy. This matches equation (11) of the paper verbatim.

2.5 Closure: Turbulence Model

The modified turbulent viscosity $\tilde{\nu}$ obeys the Spalart–Allmaras equation with the DDES length-scale modification, closed by `SpalartAllmarasDDES` in OpenFOAM.

Chapter 3

Geometry and Meshing

3.1 Geometry Definition

Table 3.1: Geometry dimensions (matches Eames 2022 Fig. 1).

Component	Parameter	Value
Main pipe	Diameter d_1	460 mm
	Length	9200 mm ($= 20\,d_1$)
	Axis	Z-direction
Branch pipe	Diameter d_2	115 mm ($\beta = 0.25$)
	Length	1380 mm ($\geq 10\,d_2$)
	Axis	$+Y$-direction
	Junction	$Z = 4600$ mm ($= 10\,d_1$)
	Injection angle	90° (top-side)

3.2 Step 1: Generate the STL

The Python script `generateSTL.py` creates four named STL surfaces in `constant/triSurface/`:

1. `wall.stl` — combined pipe wall surfaces.
2. `main_inlet.stl` — cap at $Z = 0$.
3. `outlet.stl` — cap at $Z = 9.2$ m.
4. `branch_inlet.stl` — cap at the branch tip.

```
pip install numpy numpy-stl
python3 generateSTL.py
```

3.3 Step 2: Background Mesh (`blockMesh`)

Table 3.2: `blockMeshDict` extents (background hex mesh).

Direction	Min (m)	Max (m)
X	-0.30	$+0.30$
Y	-0.30	$+1.50$
Z	-0.10	$+9.30$

Cell count $30 \times 90 \times 470$ gives a background size $\sim 20\,\text{mm}$. `snappyHexMesh` then refines to level 3 inside the junction BOI (a 690 mm-radius sphere centred on the branch).

3.4 Steps 3–4: Features + `snappyHexMesh`

```
blockMesh
surfaceFeatureExtract
snappyHexMesh -overwrite
```

The `snappyHexMeshDict` performs:

- **Castellate:** cell splitting at refinement levels $(2, 3)$ on pipe walls, plus uniform level 3 inside the junction BOI.
- **Snap:** implicit + explicit feature snap (10 iterations).
- **Layers:** 5 prismatic layers at the wall, expansion ratio 1.2, final layer 30 % of the cell size.

Target cell count: **2.8–4.8 M**, matching the paper.

3.4.1 Boundary Patches

Table 3.3: Patches created by `snappyHexMesh`.

Patch	Type	Description
`main_inlet`	`patch`	$Z = 0$ cap
`branch_inlet`	`patch`	Branch tip
`outlet`	`patch`	$Z = 9.2\,\text{m}$ cap
`wall`	`wall`	All pipe walls

Chapter 4

Physical Models and Properties

4.1 Turbulence Model (`constant/momentumTransport`)

constant/momentumTransport

```
simulationType  LES;

LES
{
    LESModel        SpalartAllmarasDDES;
    turbulence      on;
    printCoeffs     on;
    delta           cubeRootVol;

    SpalartAllmarasDDESCoeffs
    {
        curvatureCorrection yes;
    }
}
```

DES/DDES models are classified as LES in OpenFOAM. The `SpalartAllmarasDDES` closure automatically switches between RANS (in attached boundary layers) and LES (in separated, shedding regions of the junction wake).

4.1.1 Turbulent Schmidt Number

constant/turbulenceProperties

```
Sct     0.5;
```

$Sc_t = 0.5$ (versus the OpenFOAM default of 0.7) aligns with the companion Fluent case.

4.2 Thermophysical Properties

The binary H_2/CH_4 mixture is defined via JANAF thermodynamics + Sutherland transport + perfect-gas equation of state:

constant/thermophysicalProperties (header)

```
thermoType
{
    type            heRhoThermo;
    mixture         reactingMixture;
    transport       sutherland;
    thermo          janaf;
    energy          sensibleEnthalpy;
    equationOfState perfectGas;
    specie          specie;
}

inertSpecie CH4;
species     ( H2 CH4 );
```

4.2.1 Species Properties (NIST / Paper Verification)

Table 4.1: Species properties. The JANAF coefficients reproduce the paper's constant-c_p values to within 2 % at $T = 284\,\mathrm{K}$.

Property	**CH_4**	**H_2**	**Source**
Molecular weight (kg/kmol)	16.043	2.016	NIST
c_p at 284 K (J/kg/K) (JANAF)	2193	14 244	this work
c_p at 284 K (J/kg/K) (paper)	2231.5	14 320	Eames 2022
Dynamic viscosity at 284 K (Pa s)	1.25×10^{-5}	9.0×10^{-6}	NIST
Sutherland A_s (Pa s/K$^{1/2}$)	1.17×10^{-6}	6.69×10^{-7}	fitted
Sutherland T_s (K)	164	72	standard

The JANAF polynomials are marginally *more accurate* than the paper's constant-c_p assumption because they capture the weak temperature dependence of c_p across the 284–293 K inlet temperature difference.

4.3 Chemistry & Combustion (Disabled)

constant/chemistryProperties

```
chemistryType { chemistrySolver none; }
chemistry     off;
```

constant/combustionProperties

```
combustionModel none;
```

constant/reactions

```
species   ( H2 CH4 );
reactions ();
```

This is the “both combustion and reactions switched off” configuration stated in the paper.

4.4 Gravity

constant/g

```
dimensions  [0 1 -2 0 0 0 0];
value       (0 -9.81 0);
```

With top-side injection, the pipe axis is along Z and gravity acts along $-Y$. The $\rho : 8$ density contrast between CH_4 and H_2 generates the stratification that dominates the wall-concentration map.

Chapter 5

Boundary and Initial Conditions

5.1 Power-Law Inlet Velocity Profile

Eames et al. impose a fully-developed turbulent pipe profile at both inlets:

$$u_r = u^* \left[1 - \left(\frac{2r}{d^*}\right)^2\right]^{1/n}, \qquad n = 7 \tag{5.1}$$

where r is the radial distance from the pipe centreline and d^*, u^* are the pipe diameter and centreline speed. In OpenFOAM this is implemented with a `codedFixedValue` BC whose kernel is compiled at the first run:

0/U (main_inlet excerpt)

```
main_inlet
{
    type    codedFixedValue;
    value   uniform (0 0 10);
    name    mainInletPowerLaw;
    code
    #{
        const vectorField& Cf = this->patch().Cf();
        vectorField& field = *this;
        const scalar d    = 0.46,  uMax = 10.0,  n = 7.0;
        forAll(Cf, i)
        {
            const scalar r   = Foam::sqrt(sqr(Cf[i].x())
                                          + sqr(Cf[i].y()));
            const scalar arg = Foam::max(1.0 - sqr(2*r/d), scalar
                (0));
            field[i] = vector(0, 0, uMax*Foam::pow(arg, 1.0/n));
        }
    #};
}
```

The branch inlet uses the same formula with $d^* = 0.115\,\mathrm{m}$, $u^* = 32\,\mathrm{m/s}$ along $-Y$, and radius measured from the branch axis (which passes through $(0,\, y,\, 4.6)$).

5.1.1 Branch Velocity Derivation

Using the paper's definition of the volumetric dilution factor:

$$\mathcal{D} = \left(\frac{d_2}{d_1}\right)^2 \frac{u_2}{u_1} = (0.25)^2 \frac{u_2}{10}$$

For $\mathcal{D} = 0.2$ this gives $u_2 = 32\,\mathrm{m/s}$, matching equations (2)–(7) of the paper.

5.2 Pressure (`0/p` and `0/p_rgh`)

`rhoReactingBuoyantFoam` solves for $p_{\mathrm{rgh}} = p - \rho\,\mathbf{g}\cdot\mathbf{h}$ and reconstructs p internally. All physically meaningful pressure BCs therefore live in `0/p_rgh`:

Table 5.1: Pressure boundary conditions.

Patch	`p_rgh` type	Value (Pa)	Notes
`main_inlet`	`fixedFluxPressure`	—	Balances body force
`branch_inlet`	`fixedFluxPressure`	—	Balances body force
`outlet`	`prghPressure`	6 900 000	Hydrostatic removed
`wall`	`fixedFluxPressure`	—	Balances body force

The `prghPressure` BC at the outlet takes the *absolute* outlet pressure (6.9 MPa) and automatically subtracts $\rho\,\mathbf{g}\cdot\mathbf{h}$, so the stored p_{rgh} field is what the solver actually works with.

`0/p` is stored with `type calculated` everywhere because it is purely derived from p_{rgh}.

5.3 Temperature, Species, Turbulence

Table 5.2: Scalar boundary conditions.

Patch	T		Y_{H_2}		$\tilde{\nu}$
	type	value (K)	type	value	
`main_inlet`	`fixedValue`	284	`fixedValue`	0	8×10^{-7}
`branch_inlet`	`fixedValue`	293	`fixedValue`	1	8×10^{-7}
`outlet`	`inletOutlet`	284	`inletOutlet`	0	`inletOutlet`
`wall`	`zeroGradient`	—	`zeroGradient`	—	`fixedValue 0`

The walls are **adiabatic** (`zeroGradient` on T) to match the paper, which does not discuss wall heat transfer.

5.4 Initialisation (`setFieldsDict`)

A cylindrical region around the branch is patched with pure H_2 and the branch velocity, to give the solver a warm start and avoid spending many time steps filling the branch from pure methane:

system/setFieldsDict (region)

```
cylinderToCell
{
    p1      (0 0.20 4.6);
    p2      (0 1.40 4.6);
    radius  0.06;

    fieldValues
    (
        volScalarFieldValue H2  1
        volScalarFieldValue CH4 0
        volVectorFieldValue U   (0 -32 0)
    );
}
```

Chapter 6

Solution Methods and Controls

6.1 Pressure–Velocity Coupling

system/fvSolution (PIMPLE block)

```
PIMPLE
{
    nOuterCorrectors    1;      // = PISO
    nCorrectors         2;
    nNonOrthogonalCorrectors 1;
    momentumPredictor   yes;
}
```

Setting `nOuterCorrectors=1` reduces PIMPLE to the PISO algorithm, matching the companion Fluent case exactly.

6.2 Discretisation Schemes (`system/fvSchemes`)

Table 6.1: Convection scheme mapping (Fluent ↔ OpenFOAM).

Term	**OpenFOAM**	**Fluent Equivalent**
`div(phi,U)`	`Gauss LUST grad(U)`	Bounded Central Differencing
`div(phi,H2)`	`Gauss limitedLinear 1`	Second Order Upwind
`div(phi,h)`	`Gauss limitedLinear 1`	Second Order Upwind
`div(phi,nuTilda)`	`Gauss limitedLinear 1`	Second Order Upwind
`div(phiv,p)`	`Gauss limitedLinear 1`	(not exposed in Fluent)
Time (`ddtSchemes`)	`backward`	Bounded 2nd-order implicit
Gradient (`gradSchemes`)	`Gauss linear`	Least Squares Cell Based
Laplacian	`Gauss linear corrected`	(non-orthogonality correction)

Why LUST? LUST blends 75 % linear central difference with 25 % linearUpwind. It is the low-dissipation scheme that OpenFOAM developers specifically recommend for LES/DES momentum convection, and is the closest equivalent to Fluent's Bounded Central Differencing.

6.3 Under-Relaxation Factors

Table 6.2: Under-relaxation factors (matching Fluent).

Variable	**Factor**
Pressure (`p_rgh`)	0.3
Momentum (`U`)	0.7
Turbulent viscosity (`nuTilda`)	0.8
Species (`H2`, `CH4`)	0.9
Enthalpy (`h`)	1.0

Chapter 7

Run Setup and Monitors

7.1 Time-Stepping

Table 7.1: Run calculation parameters.

Parameter	Value
Application	`rhoReactingBuoyantFoam`
Time step, Δt	2×10^{-4} s
End time	5 s (= 25 000 steps)
Time statistics start	4 s (paper averages $t = 4-5$ s)
Write interval	0.1 s (every 500 steps)

CFL ≈ 1.3 at the junction (based on $u_2 = 32$ m/s and the refined cell size ~ 5 mm).

7.2 Monitors (Function Objects)

All monitors are declared in `controlDict` under `functions`; output lands in `postProcessing/`.

7.2.1 Residuals, Courant Number, y+

Automatic diagnostic function objects:

- `residuals` — per-timestep residuals for $U,\ p_{\mathrm{rgh}},\ h,\ \tilde{\nu},\ H_2$.
- `Co` — maximum Courant number at every write.
- `yPlus` — wall y^+ at every write, for mesh quality verification.

7.2.2 Mass Balance

Three `surfaceFieldValue` monitors summing ϕ (mass flux) over `main_inlet`, `branch_inlet` and `outlet`:

$$\text{imbalance} = \dot{m}_{\mathrm{out}} - \dot{m}_{\mathrm{main}} - \dot{m}_{\mathrm{branch}}$$

which should remain below 1 % of either inlet.

7.2.3 Mixing Statistics

- `outletH2` — mass-weighted average Y_{H_2} at the outlet. Should approach $V_R M_{\mathrm{H}_2}/M_{\mathrm{CH}_4}$ (e.g. ~ 0.0187 for $\mathcal{D}=0.2$).
- `wallH2` — maximum Y_{H_2} on the `wall` patch (reproduces Eames Fig. 5).
- `wallH2Mean` — area-averaged Y_{H_2} on the `wall` patch.

7.2.4 Wall Probes (Top + Bottom + Sides)

Table 7.2: Wall probes (reproduces Eames Fig. 5, Fig. 7).

Position	**Locus**	x (m)	y (m)	z (m)
$z/d_1 = 2$	top	0	+0.23	5.52
$z/d_1 = 4$	top	0	+0.23	6.44
$z/d_1 = 6$	top	0	+0.23	7.36
$z/d_1 = 8$	top	0	+0.23	8.28
$z/d_1 = 2$	bottom	0	−0.23	5.52
$z/d_1 = 4$	bottom	0	−0.23	6.44
$z/d_1 = 6$	bottom	0	−0.23	7.36
$z/d_1 = 8$	bottom	0	−0.23	8.28
$z/d_1 = 2$	side(+)	+0.23	0	5.52
$z/d_1 = 2$	side(-)	−0.23	0	5.52
$z/d_1 = 4$	side(+)	+0.23	0	6.44
$z/d_1 = 4$	side(-)	−0.23	0	6.44

The **top vs. bottom** comparison is the central asymmetry predicted by the paper and must be resolved by the probes.

7.2.5 Hydrogen Volume Fraction (Coded Field)

A `coded` function object computes and writes ϕ_{H_2} at every write step:

$$\phi_{H_2} = \frac{Y_{H_2}/M_{H_2}}{Y_{H_2}/M_{H_2} + (1 - Y_{H_2})/M_{CH_4}} \qquad (M \text{ in kg/kmol}) \tag{7.1}$$

```
phiH2
{
    type  coded;
```

```
    name  phiH2Compute;
    writeControl writeTime;
    codeExecute
    #{
        const volScalarField& Y_H2 =
            mesh().lookupObject<volScalarField>("H2");
        volScalarField phiH2( ..., (Y_H2/2.016)
              / (Y_H2/2.016 + (1.0-Y_H2)/16.043) );
        phiH2.write();
    #};
}
```

7.2.6 Time-Averaged Fields

- `UMean`, `UPrime2Mean` — mean velocity and Reynolds stresses.
- `pMean` — mean absolute pressure.
- `H2Mean`, `H2Prime2Mean` — mean H_2 mass fraction and its variance (for COV calculation).
- `TMean` — mean temperature.

7.3 Full Workflow

```
./Allrun
```

Executes the full pipeline:

1. `python3 generateSTL.py` — build STLs.
2. `blockMesh` — background mesh.
3. `surfaceFeatureExtract` — feature edges.
4. `snappyHexMesh -overwrite` — conformal mesh.
5. `setFields` — patch branch with pure H_2.
6. `decomposePar` — 8-way scotch decomposition.
7. `mpirun -np 8 rhoReactingBuoyantFoam -parallel`.
8. `reconstructPar` — gather for post-processing.

Chapter 8

Validation and Expected Results

8.1 Validation Strategy

Eames et al. validate their OpenFOAM setup against the **OECD/NEA–Vättenfall T-junction benchmark** [1], characterised by $Q_1/Q_2 = 1,\ d_1/d_2 = 1$ and a matched-Froude cold/hot water mixing experiment.

For consistency, the same benchmark should be used for OpenFOAM validation here:

1. Rebuild the case with **isothermal water-like properties** using the same T-junction geometry but at $d_1/d_2 = 1$ and $Q_1/Q_2 = 1$.
2. Match the **Froude number** $Fr = u_2/\sqrt{|\rho_1 - \rho_2|\, g/\rho_1}$ reported in [1].
3. Compare the mean and RMS streamwise velocity in the centreline vertical plane at $z = 1.86\, d_1$ to experimental profiles.

Paper Fig. 2(b) confirms good agreement at 2 M cells; finer meshes are unnecessary.

8.2 Expected Behaviour (Production Run, $\mathcal{D} = 0.2$)

Table 8.1 summarises the *target values* that the converged simulation should reproduce.

Table 8.1: Expected results for the baseline case ($\mathcal{D} = 0.2$, $\beta = 0.25$, top-side injection).

Quantity	**Target**
Outlet H_2 volume fraction V_R	≈ 0.149 (paper Eqs. 2,6)
Outlet H_2 mass fraction Y_{H_2}	≈ 0.0187
Mass-balance residual $\lvert\dot{m}_{\text{out}} - \dot{m}_{\text{main}} - \dot{m}_{\text{branch}}\rvert$	$< 1\,\%$ of either inlet
Max wall y^+	< 100 (all-y^+ wall fn.)
Max Courant number	≈ 1.3
Momentum ratio $\mathcal{M}_R$	~ 0.3 (weak penetration)
Max wall V_{Rm} at $z/d_1 = 8$	$\approx 1.6 \cdot V_{R,T}$ (Fig. 7b)
Shedding frequency $f_s \sim u_1 \cdot St/d_2$	$\approx 17\,\text{Hz}$ (St ~ 0.2)

8.3 Paper Physical Findings to Reproduce

The simulation should, qualitatively and quantitatively, reveal:

1. **Stratification.** For $\mathcal{D} < 0.3$, H_2 is confined to the upper portion of the pipe by buoyancy. Top-wall H_2 concentration $> 40\,\%$ for up to $8\,d_1$ downstream.

2. **Weak jet penetration.** The low-inertia H_2 jet is deflected by the main flow and fails to traverse the main pipe for $\mathcal{D} \lesssim 0.2$.

3. **Periodic shedding.** At $\mathcal{D} = 0.1$ puffs separated by $\sim 0.8\,\mathrm{m}$ ($St \sim 0.2$).

4. **Asymmetric wall exposure.** The lower pipe wall sees much less H_2 than the upper wall for $\mathcal{D} \leq 0.2$. This is the stratification signal that should appear clearly in the wall-probe time series.

Bibliography

[1] O. Braillard, R. Howard, K. Angele, A. Shams, N. Edh. "Thermal mixing in a T-junction: novel CFD-grade measurements of the fluctuating temperature in the solid wall." *Nuclear Engineering and Design*, 330:377–390, 2018.

Chapter 9

Post-Processing

9.1 In-Run Outputs

All function-object outputs land in `postProcessing/`:

Table 9.1: Runtime output directories.

Directory	Contents
`residuals/`	Solver residual history.
`Co/`, `yPlus/`	Courant number, wall y^+ snapshots.
`outletH2/`	Mass-weighted Y_{H_2} at outlet.
`outletMassFlow/`	Outlet mass-flow time series.
`mainInletMassFlow/`	Main-inlet mass-flow time series.
`branchInletMassFlow/`	Branch mass-flow time series.
`wallH2/`	Max Y_{H_2} on pipe wall vs. time.
`wallH2Mean/`	Area-averaged Y_{H_2} on wall.
`wallH2Probes/`	Point values at all 12 probes.
`fieldAverage1/`	`UMean, H2Mean, H2Prime2Mean, ...`

All are plain-text CSV-like columns (time + value) that can be plotted directly with `gnuplot`, Python or Excel.

9.2 Paraview Workflow

```
reconstructPar
paraFoam
```

9.2.1 Reproducing Paper Figures

1. **Fig. 3a (Instantaneous V_R, vertical slice):** In Paraview, load the case → apply `Slice` filter, origin $(0, 0, 4.6)$, normal $(1, 0, 0)$ (i.e. the YZ-plane). Colour by `phi_H2`

at late snapshot.

2. **Fig. 3b (Time-averaged $\overline{V_R}$):** Same slice, but colour by `H2Mean` then apply the same volume-fraction formula in a `Calculator` filter.

3. **Fig. 3c (Max V_{Rm} vs. z):** Use the `Plot Over Line` filter along the upper-wall line ($x = 0,\ y = 0.23,\ z \in [4.6, 9.2]$) colouring by `H2Mean` $\Rightarrow$ `phi_H2`.

4. **Fig. 4 (Wall map):** Apply `Extract Surface` filter on the wall boundary, then colour by the coded `phi_H2` field.

5. **Fig. 5 (Unwrapped wall):** Apply `Extract Surface` $\rightarrow$ `Clip` $\rightarrow$ unwrap using custom cylindrical-to-cartesian transform filter.

9.3 Coefficient of Variation (COV)

COV quantifies mixing uniformity on a cross-section:

$$\mathrm{COV} = \frac{\sigma_\phi}{\bar{\phi}}, \quad \sigma_\phi = \sqrt{\texttt{H2Prime2Mean}}, \quad \bar{\phi} = \texttt{H2Mean}.$$

COV = 0 denotes perfect mixing, COV = 1 denotes complete segregation.

9.4 Mesh Independence (GCI)

Per the paper, run three mesh refinement levels (1.0, 2.0, 4.0 M cells) and compute the **Grid Convergence Index** (Roache 1994) on the converged outlet V_R:

$$p = \frac{\ln\left|\frac{\phi_3-\phi_2}{\phi_2-\phi_1}\right|}{\ln r}, \qquad \mathrm{GCI}_{\mathrm{fine}} = F_s \frac{|(\phi_2-\phi_1)/\phi_1|}{r^p - 1}, \quad F_s = 1.25.$$

A GCI $< 2\,\%$ at the finest level confirms mesh independence.

Chapter 10

Quick-Start Summary

The complete workflow from a clean directory:

1. **Install Python deps:**

```
pip install numpy numpy-stl
```

2. **Run the full pipeline:**

```
./Allrun
```

3. **Monitor the run:**

```
tail -f log.rhoReactingBuoyantFoam
```

4. **Post-process:**

```
reconstructPar
paraFoam
```

Pre-flight checklist:

- ✓ `WM_PROJECT_DIR` set (i.e. OpenFOAM environment sourced).
- ✓ STLs present in `constant/triSurface/`.
- ✓ `thermophysicalProperties` species list matches `0/H2` and `0/CH4`.
- ✓ Patch names in all `0/` files match the `snappyHexMeshDict` surface names.
- ✓ `decomposeParDict` `numberOfSubdomains` = `mpirun -np` count.
- ✓ `controlDict` `application` = `rhoReactingBuoyantFoam`.
- ✓ `fieldAverage1` `timeStart` = 4.0 s.